R Probability and Statistics Cheat Sheet

Table of contents

R Probability and Statistics Cheat Sheet 1
- Minimum R syntax 1
  - Operators 2
  - Common mathematical functions 2
- Creating and selecting data 2
- Describing a sample 3
- Quick plots 3
- Reading tabular data 3
- R distribution functions: `d`, `p`, `q`, `r` 4
  - Tail and interval patterns 4
  - Course distribution lookup 5
- Reliability and hazard 5
- Choosing and fitting a distribution to data 6
  - 1. Choose plausible distributions 6
  - 2. Estimate parameters and check the fit in R 7
  - 3. Use the fitted distribution 7
- Model estimates from complete data 7
- Normal, t and CLT calculations 8
- Random sampling and reproducible simulation 8
- Custom functions and conditions 9
- Kaplan–Meier estimate for right-censored data 9
- Printing a requested number of decimals 9
- Fast error checks 9

This is the working subset of R used in the probability notes. Start with the syntax section, then use the remaining sections as lookup tables and calculation patterns.

In RStudio, place code in an R script and run the current line or selection with `Ctrl+Enter` (`Cmd+Enter` on macOS). A bare object name displays its value:

```
life_time
```

Minimum R syntax

```
# A comment starts with #
x <- 5                        # assign 5 to x
answer <- sqrt(x)             # call a function
pnorm(2, mean = 0, sd = 1)    # named function arguments

values <- c(4.8, 3.9, 2.2)    # combine values into a vector
values[1]                     # first value; R counts from 1
values[values > 3]            # values satisfying a condition
```

Syntax	Meaning
`name <- value`	Store a value. Prefer `<-` for assignment.
`function_name(arguments)`	Call a function.
`c(a, b, c)`	Combine values into a vector.

Syntax	Meaning
`x[i]`	Select element `i` the first element is `x[1]`.
`x[a:b]`	Select positions `a` through `b`.
`data$time`	Select the `time` column of a data frame.
`package::function()`	Use a function without first calling `library(package)`.
`# comment`	R ignores the rest of the line.
`?pnorm`	Open help for a function.

R is case-sensitive: `x`, `X` and `mean_x` are different names. A decimal uses a full stop, and scientific notation such as `2.5e-3` means 2.5×10^{-3}.

Operators

Operation	R
Add, subtract, multiply, divide	`+`, `-`, `*`, `/`
Power	`x^2`
Integer sequence	`1:10`
Equal / not equal	`==`, `!=`
Less / greater	`<`, `>`, `<=`, `>=`
And / or / not	`&`, `|`, `!`
Is one of these values	`x %in% c(1, 3, 5)`

Use parentheses to make the order explicit:

```
standard_error <- sigma / sqrt(n)
z <- (boundary - mu) / standard_error
```

Common mathematical functions

```
sqrt(x)          # square root
abs(x)           # absolute value
log(x)           # natural logarithm, ln(x)
exp(x)           # e^x
gamma(x)         # Gamma(x)
factorial(n)     # n!
choose(n, k)     # n choose k
ceiling(x)       # round up
round(x, 3)      # round to 3 decimal places
```

R applies arithmetic and most functions to every element of a vector:

```
x <- c(1, 2, 3)
x^2                 # 1, 4, 9
(x - mean(x))^2     # squared deviation of each observation
```

Creating and selecting data

```
x <- c(10, 17, 24, 30, 14)
1:5
seq(0, 1, by = 0.2)
rep(4, times = 3)

length(x)
```

```r
sort(x)
x[2]
x[c(1, 3, 5)]
x[x >= 17]
```

Useful special values are `NA` for a missing value, `NaN` for an undefined numeric result and `Inf` for infinity. Many summaries need `na.rm = TRUE` when missing values are present.

```r
x <- c(2, 4, NA, 8)
is.na(x)
mean(x, na.rm = TRUE)
```

Describing a sample

```r
life_time <- c(4.8, 3.9, 2.2, 5.1, 4.6,
               5.3, 6.3, 4.3, 6.8, 4.7)

length(life_time)
mean(life_time)
median(life_time)
var(life_time)
sd(life_time)
min(life_time)
max(life_time)
range(life_time)
quantile(life_time)
summary(life_time)
```

`var(x)` and `sd(x)` use denominator $n-1$. For the normal maximum-likelihood estimate of σ, the denominator is n:

```r
sigma_sample <- sd(life_time)
sigma_mle <- sqrt(mean((life_time - mean(life_time))^2))
```

Quick plots

```r
hist(life_time,
     main = "Histogram of lifetimes",
     xlab = "Lifetime (years)")

boxplot(life_time, horizontal = TRUE)
stem(life_time)
plot(life_time, type = "o")
```

To draw a mathematical function:

```r
f_X <- function(x) {
  ifelse(x <= 0 | x >= 80, 0,
         ifelse(x <= 40, x / 1600, (80 - x) / 1600))
}

curve(f_X, from = -5, to = 85,
      xlab = "x", ylab = "f_X(x)")
```

`par(mfrow = c(1, 2))` divides the plotting window into one row and two columns. In `curve()`, `add = TRUE` draws on the current plot.

Reading tabular data

These commands assume the required packages are already available.


```r
# CSV with column headings
failure_data <- read.csv("exa1.csv")

# Whitespace- or tab-separated text with column headings
microwaves <- read.table("microwaves.txt", header = TRUE)

# Excel workbook
streetlamps <- readxl::read_excel("streetlamps.xlsx")

head(failure_data)
names(failure_data)
str(failure_data)
summary(failure_data)

failure_data$time
failure_data$status
failure_data[failure_data$status == 1, ]
```

Relative file paths start from `getwd()`. Use `getwd()` to check the current working directory and prefer project-relative paths over changing it in a script.

R distribution functions: `d`, `p`, `q`, `r`

R uses the same four prefixes for nearly every distribution.

Prefix	Question answered	Normal example
`d`	What is the PMF/PDF value at x?	`dnorm(x, mean, sd)`
`p`	What is $P(X \le x)$?	`pnorm(x, mean, sd)`
`q`	Which x has a given lower-tail probability?	`qnorm(p, mean, sd)`
`r`	Generate random observations.	`rnorm(N, mean, sd)`

“Given a boundary, use `p`. Given a probability and asked for the boundary, use `q`. Use `d` only for a PMF/PDF height, not an accumulated probability.”

Tail and interval patterns

```r
# P(X <= x)
pnorm(x, mean = mu, sd = sigma)

# P(X > x): use the upper tail directly
pnorm(x, mean = mu, sd = sigma, lower.tail = FALSE)

# P(a < X <= b)
pnorm(b, mean = mu, sd = sigma) -
  pnorm(a, mean = mu, sd = sigma)

# x such that P(X <= x) = p
qnorm(p, mean = mu, sd = sigma)

# x such that P(X > x) = alpha
qnorm(alpha, mean = mu, sd = sigma, lower.tail = FALSE)
```

For a discrete variable, the CDF includes its boundary:

```r
# X ~ Binomial(10, 0.08)
pbinom(1, size = 10, prob = 0.08)                    # P(X <= 1)
```

```r
pbinom(1, size = 10, prob = 0.08, lower.tail = FALSE) # P(X > 1)
pbinom(4, size = 10, prob = 0.08) -
  pbinom(1, size = 10, prob = 0.08)                   # P(2 <= X <= 4)
```

Course distribution lookup

Replace the first letter with `d`, `p`, `q` or `r`. For an `r` function, the first argument is the number of observations `N` rather than `x` or a probability.

Distribution	R pattern	Parameter note
Discrete uniform	`sample(values, N, replace = TRUE)`	Each listed value is equally likely.
Continuous uniform	`punif(x, min = a, max = b)`	Bounds are (a,b).
Binomial	`pbinom(x, size = n, prob = p)`	Trials n, success probability p.
Poisson	`ppois(x, lambda = mu)`	`lambda` is the expected count for the interval.
Normal	`pnorm(x, mean = mu, sd = sigma)`	`sd` is σ, not σ².
Student's t	`pt(x, df = nu)`	Degrees of freedom `df`.
Lognormal	`plnorm(x, meanlog = mu, sdlog = sigma)`	`mu` and `sigma` describe `log(X)`.
Weibull	`pweibull(x, shape = beta, scale = eta)`	Shape β, characteristic life η.
Exponential	`pexp(x, rate = lambda)`	Mean and scale are $\frac{1}{\lambda}$.
Gamma	`pgamma(x, shape = beta, rate = lambda)`	Scale θ is $\frac{1}{\lambda}$. Do not supply both.
Beta	`pbeta(x, shape1 = alpha, shape2 = beta)`	Support is $0 \leq x \leq 1$.

Examples of the other prefixes:

```r
dweibull(t, shape = beta, scale = eta)
qgamma(0.78, shape = beta, rate = lambda)
rlnorm(1000, meanlog = mu, sdlog = sigma)
```

Reliability and hazard

For a lifetime T, reliability is the upper tail:

```r
# R(t) = P(T > t)
R_exponential <- pexp(t, rate = lambda, lower.tail = FALSE)
R_weibull <- pweibull(t, shape = beta, scale = eta,
                      lower.tail = FALSE)
R_lognormal <- plnorm(t, meanlog = mu, sdlog = sigma,
                      lower.tail = FALSE)
```

Common reliability formulas:

```r
# Exponential
h_exponential <- lambda
mttf_exponential <- 1 / lambda

# Weibull
h_weibull <- (beta / eta) * (t / eta)^(beta - 1)
mttf_weibull <- eta * gamma(1 + 1 / beta)
```



```r
sd_weibull <- eta * sqrt(
  gamma(1 + 2 / beta) - gamma(1 + 1 / beta)^2
)

# Independent systems at one time t
R_series <- prod(component_reliabilities)
R_parallel <- 1 - prod(1 - component_reliabilities)
```

Choosing and fitting a distribution to data

To "assign a distribution" to data, choose a probability model for the variable and estimate its parameters from the observations. The data do not uniquely determine a distribution: use what the variable measures, its possible values, and the observed shape. If a question specifies the distribution, use that model.

1. Choose plausible distributions

What the data represent	Candidate model	What to check
Successes out of a fixed number of trials	Binomial	Independent trials with the same success probability; known trial count.
Event counts in equal exposure intervals	Poisson	Independent events at a constant rate; variance roughly equal to the mean.
Continuous measurements with roughly symmetric, bell-shaped variation	Normal	No strong skew or outliers; negative model values must be acceptable or negligible.
Positive measurements with right skew and multiplicative variation	Lognormal	`log(x)` is approximately normal.
Nonnegative waiting times or lifetimes	Exponential	Approximately constant failure/ event rate; standard deviation roughly equals the mean.
Positive lifetimes with a changing failure rate	Weibull	Can represent decreasing, constant or increasing hazard.
Positive, right-skewed measurements or waiting times	Gamma	Flexible positive model; compare its fit with other plausible candidates.
Continuous proportions strictly between 0 and 1	Beta	Exact zeros and ones need separate treatment.
Values within known bounds with no favoured region	Uniform	Equal density throughout the interval is justified by the process.

A histogram is a starting point, not proof. Check units, missing values and recording errors first. Look at `plot(x, type = "o")` if observations are in time order: trends or dependence can invalidate a single independent, identically distributed model. Do not remove valid outliers just to improve fit.

2. Estimate parameters and check the fit in R

This example uses complete, uncensored observations and fits a normal model. The estimates describe individual observations, so use the fitted standard deviation, not the standard error of the mean.

```
x <- life_time
mu_hat <- mean(x)
sigma_hat <- sqrt(mean((x - mu_hat)^2))   # normal MLE

# Compare the fitted density with the observed histogram
hist(x, probability = TRUE,
     main = "Normal fit to lifetimes", xlab = "Lifetime (years)")
curve(dnorm(x, mean = mu_hat, sd = sigma_hat),
      add = TRUE, col = "red", lwd = 2)

# Normal Q-Q plot: points should be approximately on a straight line
qqnorm(x)
qqline(x, col = "red")

# Compare the empirical CDF with the fitted CDF
plot(ecdf(x), main = "Observed and fitted CDF", xlab = "Lifetime (years)")
curve(pnorm(x, mean = mu_hat, sd = sigma_hat),
      add = TRUE, col = "red", lwd = 2)
```

Systematic curvature in a Q-Q plot suggests a mismatch; inspect the tails as well as the centre. For a lognormal candidate with all `x > 0`, use `qqnorm(log(x))` and `qqline(log(x))`, then estimate parameters on the log scale as shown below. Small samples provide limited evidence about distribution shape. Choose a model that matches both the data and the generating process; a good visual fit does not prove the model is correct.

3. Use the fitted distribution

```
# Under the fitted normal model: probability lifetime exceeds 6 years
pnorm(6, mean = mu_hat, sd = sigma_hat, lower.tail = FALSE)

# Fitted 90th percentile
qnorm(0.90, mean = mu_hat, sd = sigma_hat)

# Observed fraction exceeding 6, for comparison
mean(x > 6)
```

These fitted probabilities treat the estimated parameters as fixed; they do not include parameter uncertainty. Calling `rnorm()` generates new simulated data, and calling `dnorm()` evaluates a density; neither fits a distribution. The usual one-sample `ks.test()` p-value is not valid when its distribution parameters were estimated from the same sample.

Model estimates from complete data

```
x <- c(10, 17, 24, 30, 14, 18, 15, 13, 21)

# Normal model
mu_normal <- mean(x)
sigma_normal_mle <- sqrt(mean((x - mu_normal)^2))
sigma_normal_sample <- sd(x)

# Lognormal model: estimate on the log scale
```



```
log_x <- log(x)
mu_lognormal <- mean(log_x)
sigma_lognormal_mle <- sqrt(mean((log_x - mu_lognormal)^2))

# Exponential model
lambda_exponential <- 1 / mean(x)
```

For exponential data with right-censoring, let `status` be 1 for failure and 0 for censored:

```
failures <- sum(failure_data$status)
total_time <- sum(failure_data$time)

lambda_hat <- failures / total_time
mttf_hat <- total_time / failures
```

Do not apply `sd()` directly to censored recorded times as though every item failed.

Normal, t and CLT calculations

```
# Sampling distribution of a mean when population sigma is known
standard_error <- sigma / sqrt(n)
z <- (x_bar - mu) / standard_error

# P(X_bar > boundary)
pnorm(boundary, mean = mu, sd = standard_error,
      lower.tail = FALSE)

# t statistic when sigma is unknown
t_statistic <- (x_bar - mu) / (s / sqrt(n))
pt(t_statistic, df = n - 1, lower.tail = FALSE)

# Required n for two-sided error E and confidence 1 - alpha
z_critical <- qnorm(1 - alpha / 2)
n_required <- ceiling((z_critical * sigma / E)^2)
```

For $T \sim \text{Weibull}(\beta, \eta)$, calculate the population mean and standard deviation before applying the CLT to a sample mean:

```
mu_T <- eta * gamma(1 + 1 / beta)
sigma_T <- eta * sqrt(
  gamma(1 + 2 / beta) - gamma(1 + 1 / beta)^2
)
standard_error <- sigma_T / sqrt(n)
```

Random sampling and reproducible simulation

```
set.seed(5507)

rnorm(20, mean = 10, sd = 5)
rexp(20, rate = 2)
sample(1:6, size = 20, replace = TRUE)

n <- 40
repetitions <- 10000
sample_means <- replicate(
  repetitions,
  mean(rexp(n, rate = 2))
)
```

```r
hist(sample_means, freq = FALSE,
     main = "Sampling distribution of the mean",
     xlab = "Sample mean")

curve(dnorm(x, mean = 1 / 2, sd = (1 / 2) / sqrt(n)),
      add = TRUE, col = "red")
```

`set.seed()` makes the generated sequence repeatable. Change or remove it when you want a different simulation.

Custom functions and conditions

```r
F_X <- function(x) {
  if (x < 1) {
    return(0)
  } else {
    return(1 - 2 / (x * (x + 1)))
  }
}

probability <- F_X(2.7) - F_X(2.2)
```

Use `if` when testing one condition. Use vectorised `ifelse(condition, yes, no)` when `x` may contain many values.

Kaplan–Meier estimate for right-censored data

This assumes the `survival` package is already available and `status` is 1 for failure and 0 for censored.

```r
library(survival)

fit <- survfit(Surv(time, status) ~ 1, data = failure_data)

plot(fit,
     xlab = "Time",
     ylab = "Estimated reliability R(t)",
     conf.int = TRUE)

summary(fit, times = c(20, 50, 100))
```

Printing a requested number of decimals

```r
p <- pnorm(0.21, lower.tail = FALSE)
round(p, 2)                  # numeric value rounded for display
cat(sprintf("%.2f\n", p))   # print exactly two decimal places
```

Fast error checks

- Check whether the question asks for a density, probability, quantile or random sample before choosing `d`, `p`, `q` or `r`.
- Use `lower.tail = FALSE` for an upper tail instead of manually subtracting a value very close to 1.
- For a discrete CDF, `pbinom(k, ...)` and `ppois(k, ...)` include (k).
- R's normal functions take standard deviation, not variance.
- R's exponential functions take a rate; mean $= \frac{1}{\lambda}$.
- Lognormal parameters belong to `log(X)`, not directly to `X`.

- Weibull `scale = eta` is characteristic life, not generally the mean.
- `sd(x)` uses (n-1); the normal MLE for σ uses (n).
- Do not overwrite common function names with objects such as `c`, `mean`, `sd`, `F` or `q`.